Lesson 3 — I/O Port Programming

ATmega328P / Arduino Uno | Source: lecture 03 slides, Exercise 1 and 2, peer notes

WHAT YOU WILL BE ABLE TO DO AFTER THIS LESSON

- Explain what an I/O port is and what the three registers `DDRx`, `PORTx` and `PINx` do.
- Make a pin an output, drive an LED on and off, and read a pushbutton.
- Write the same blinker in **assembly**, **C** and **Arduino**, and say why they differ.
- Calculate a delay (in clock cycles and milliseconds) and the resistor for an LED.

GIVEN TO YOU IN THE EXAM (DO NOT MEMORISE THESE)

ATmega328P Simplified Data Sheet: the bit names of every register (`DDB0`, `PORTB5`, `PINB0`…). **AVR C cheat sheet**: all C operators including bitwise `& | ^ ~ << >>` and `|= &= ^=`, data types, `_delay_ms()` / `_delay_us()`. **Arduino cheat sheet**: `pinMode`, `digitalRead`, `digitalWrite`, `delay` and the Uno board picture.
Not given, so learn them from these notes: what DDR / PORT / PIN *mean*, the port-to-Arduino pin map, the set/clear/toggle/test patterns, the assembly instructions and the delay calculation.

PRACTICAL EXAM WARNING (FROM THE LECTURER)

Even a working system loses marks if power wires are not **red (+5 V) and black (GND)**, or if the code lacks proper **names, indentation and comments**. See section L1.

0. Words and numbers you need first

Word	Plain meaning
Microcontroller (MCU)	A whole tiny computer on one chip: CPU, flash (program memory), RAM, and I/O pins. Ours is the **ATmega328P**, the black 28-pin chip on the Arduino Uno.
Bit / byte	A bit is one 0 or 1. A byte is 8 bits, for example `10110010`. Bits are numbered 7 (left) to 0 (right).
Register	A small 8-bit storage box inside the chip that has a **job**. Writing to some registers changes hardware (e.g. switches a pin on). Reading some registers tells you about hardware (e.g. a pin's voltage). `DDRB`, `PORTB`, `PINB` are registers.
Pin / port	A pin is one metal leg of the chip. A port is a group of up to 8 pins controlled together by one set of registers (Port B, C, D).
Firmware	The program stored in the chip's flash. It stays when the power is off and runs from the start every time the chip is powered or reset.
Compiler / HEX file	The compiler turns your C text into machine code. The HEX file is that machine code written in hexadecimal, ready to be uploaded into flash.
Clock, MHz, cycle	The CPU does work in steps timed by a clock. 16 MHz = 16 million steps (cycles) per second. One cycle = 62.5 ns.
HIGH / LOW	HIGH = logic 1 = about 5 V. LOW = logic 0 = about 0 V (GND). The chip treats roughly 0–1 V as LOW and 2–5 V as HIGH.
VCC / GND	VCC is the +5 V supply. GND (ground) is 0 V. **Every part of a circuit must share the same GND.**

Binary and hexadecimal (you will read and write these constantly)

Each bit has a place value. Add the values of the bits that are 1.

Hex: split the byte into two groups of 4 bits (nibbles) and write one hex digit for each.

bit	7	6	5	4	3	2	1	0
value	128	64	32	16	8	4	2	1
0b00100001	0	0	**1**	0	0	0	0	**1**

`0b00100001` = 32 + 1 = **33**. It means "bit 5 and bit 0". In C, `0b` starts a binary number, `0x` a hex number.

4 bits	hex	4 bits	hex
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F

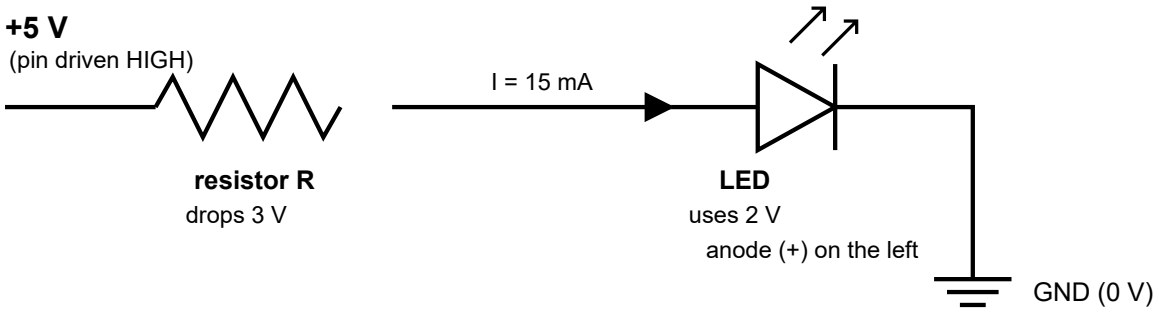
`0b0010 0001` = `0x21`. `0b1111 1011` = `0xFB`. 255 = `0xFF` = `0b11111111`.

1. Just enough electricity

You need four ideas. Think of electricity as water in pipes.

Idea	Water picture	Meaning
Voltage (V), unit volt	Water pressure	The push that makes charge move. Measured *between two points*. The Uno's supply is 5 V; ground (GND) is the 0 V reference.
Current (I), unit ampere (A), mA = 0.001 A	Flow rate of the water	Amount of charge flowing per second. Flows from the higher voltage to the lower voltage, around a closed loop.
Resistance (R), unit ohm (Ω)	A narrow pipe	Limits the current. More resistance gives less current.
Ohm's law		**V = I × R** so R = V / I

The LED and its resistor (the circuit used all through this lesson)



Same current everywhere in the loop. The 5 V is shared: 3 V across R + 2 V across the LED = 5 V.

Figure 1. Current leaves the pin (5 V), goes through R, then the LED, and returns to GND.

RESISTOR VALUE (EXERCISE 1A)

R = (V_{supply} − V_{LED}) / I_{LED} = (5 − 2) / 0.015 = **200 Ω** → use the next standard value **220 Ω** (current becomes about 13.6 mA, a little less, which is safe).

Never connect an LED without a resistor: the LED has almost no resistance of its own and would draw too much current.

Reading the resistor colour bands

Colour	Digit	Colour	Digit
Black	0	Green	5
Brown	1	Blue	6
Red	2	Violet	7
Orange	3	Grey	8
Yellow	4	White	9

Band 1 and band 2 = two digits. Band 3 = how many zeros to add. Band 4 = tolerance (ignore).

The ones you have

Bands (1, 2, 3)	Value
red, red, brown (2, 2, +0)	**220 Ω**
brown, black, red (1, 0, +00)	**1 kΩ**
brown, black, orange (1, 0, +000)	**10 kΩ**

LED direction

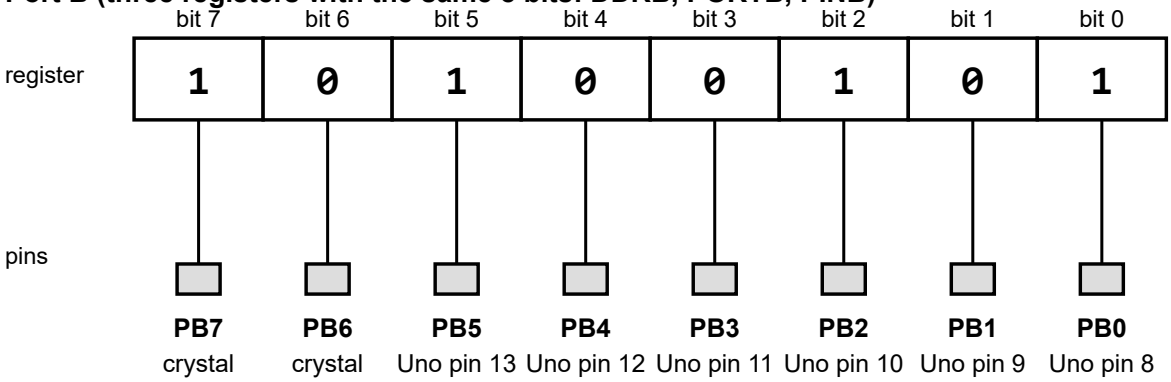
Current only flows **anode** → **cathode**. Anode = long leg. Cathode = short leg and the flat edge on the plastic. Wrong way round = does not light, so just flip it.

2. What is an I/O port?

THE BIG IDEA

An **I/O port** is an **8-bit register** that is physically wired to 8 pins of the chip. **One bit = one pin.** Write a bit and the pin voltage changes. Read a bit and you learn the voltage on the pin. That is how software touches the real world.

Port B (three registers with the same 8 bits: DDRB, PORTB, PINB)



Count the pins like an array: the rightmost bit is bit 0 = PB0. "PB5" means Port B, bit 5.
PB6 and PB7 drive the crystal on the Uno, so you cannot use them.

Figure 2. One bit of the register is wired to one pin of the chip.

Which pin is which (Arduino Uno)

Port	Bits	Arduino pins	Remember
Port D (PD)	PD0 – PD7	0 – 7	PD0/PD1 = RX/TX serial, avoid for sensors. PD2/PD3 are the external-interrupt pins. PD3, 5, 6 = PWM.
Port B (PB)	PB0 – PB5	8 – 13	**PB5 = pin 13 = the on-board LED.** PB1, 2, 3 = PWM.
Port C (PC)	PC0 – PC5	A0 – A5	Analog inputs (ADC0 – ADC5).

3. The three registers of every port (x = B, C or D)

DDRX — DATA DIRECTION REGISTER

Chooses the **direction** of each pin.
1 = output (the chip drives the pin)
0 = input (the chip listens)
Memory trick: **I**nput comes before **O**utput, just as 0 comes before 1.

PORTX — PORT OUTPUT REGISTER

If the pin is an **output**: 1 = pin HIGH (5 V), 0 = pin LOW (0 V).
If the pin is an **input**: writing 1 switches on the internal **pull-up resistor**.

PINX — PORT INPUT REGISTER

Read only. Each bit shows the actual voltage on the pin: 1 = HIGH, 0 = LOW. Use it to read buttons and digital sensors.

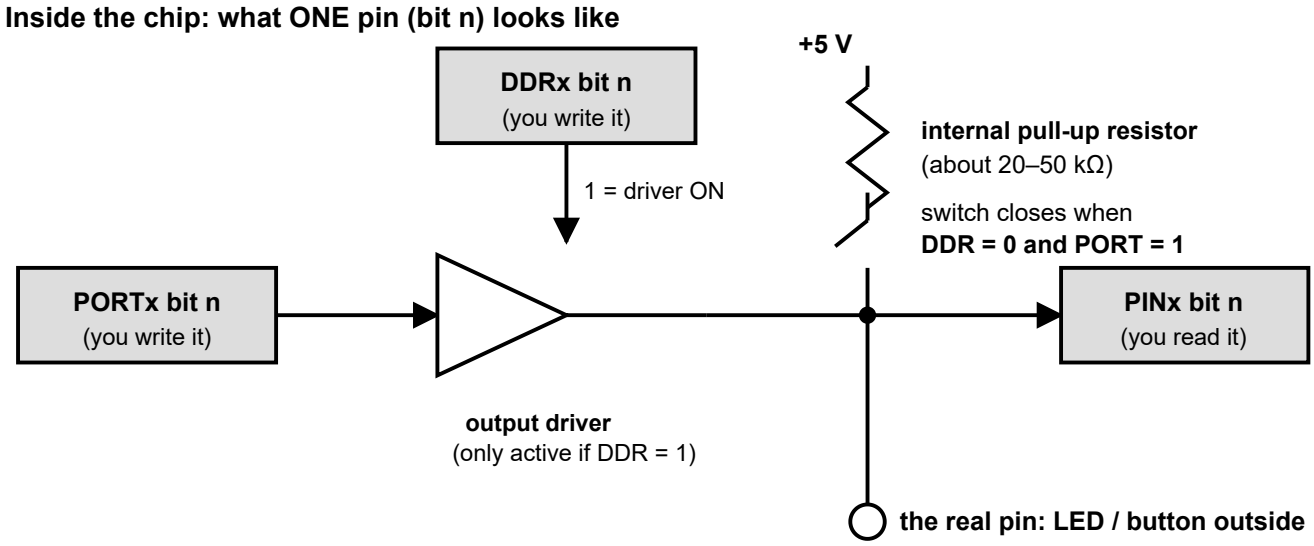


Figure 3. You write DDR and PORT; the chip's circuit drives the pin; PIN simply reports what the pin voltage really is.

4. The four things a pin can be (DDR + PORT)

DDR	PORT	The pin is	What really happens
0	**0**	**Input, floating**	Listens only. Nothing holds the voltage, so with nothing connected it picks up noise and reads random 0s and 1s.
0	**1**	**Input with pull-up**	Internal resistor ties the pin to 5 V. Reads **1** until a button pulls it to GND, then **0**. This is how you wire a button with no external resistor.
1	**0**	**Output LOW**	Chip drives the pin to 0 V. Current can flow *into* the pin (LED between 5 V and the pin would light).
1	**1**	**Output HIGH**	Chip drives the pin to 5 V. Current flows *out of* the pin (our LED circuit). Keep it below 20 mA per pin.

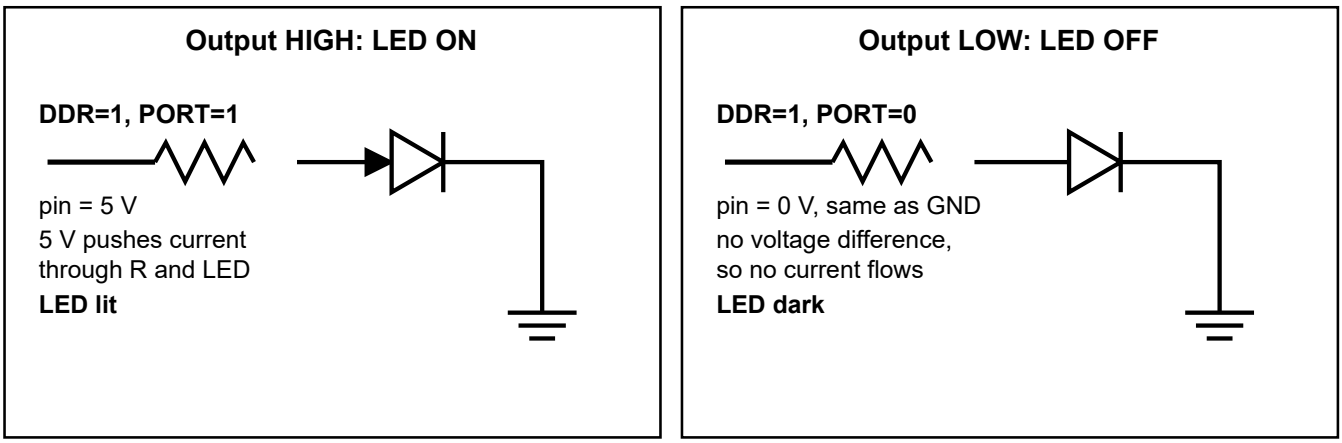


Figure 4. Why HIGH lights the LED: it creates a 5 V difference between the pin and GND.

5. Bit operations: change ONE pin without touching the others

A port has 8 pins. If you write `PORTB = 0b00000001;` you overwrite **all 8 bits** and may switch off other pins that were on. Use these four patterns instead. `n` is the bit number. `(1 << n)` builds a mask with only bit n set.

Mask building: `1 << n` means "put a 1 in position n"

	7	6	5	4	3	2	1	0	bit positions
`1 << 0`	0	0	0	0	0	0	0	**1**	= 0b00000001
`1 << 3`	0	0	0	0	**1**	0	0	0	= 0b00001000
`1 << 5`	0	0	**1**	0	0	0	0	0	= 0b00100000
`~(1 << 3)` (invert all)	**1**	**1**	**1**	**1**	0	**1**	**1**	**1**	= 0b11110111

SET bit 0 (make 1): `PORTB |= (1 << 0);` — OR with the mask

PORTB before	1	0	1	0	0	1	0	0	
mask `(1<<0)` OR	0	0	0	0	0	0	0	**1**	
PORTB after	1	0	1	0	0	1	0	**1**	only bit 0 changed. x OR 1 = 1, x OR 0 = x

CLEAR bit 0 (make 0): `PORTB &= ~(1 << 0);` — AND with the inverted mask

PORTB before	1	0	1	0	0	1	0	1	
mask `~(1<<0)` AND	**1**	**1**	**1**	**1**	**1**	**1**	**1**	0	
PORTB after	1	0	1	0	0	1	0	**0**	x AND 1 = x, x AND 0 = 0

TOGGLE bit 5 (flip): `PORTB ^= (1 << 5);` — XOR with the mask

PORTB before	1	0	1	0	0	1	0	1	
mask `(1<<5)` XOR	0	0	**1**	0	0	0	0	0	
PORTB after	1	0	**0**	0	0	1	0	1	bit 5 was 1, now 0. x XOR 1 = flip, x XOR 0 = x

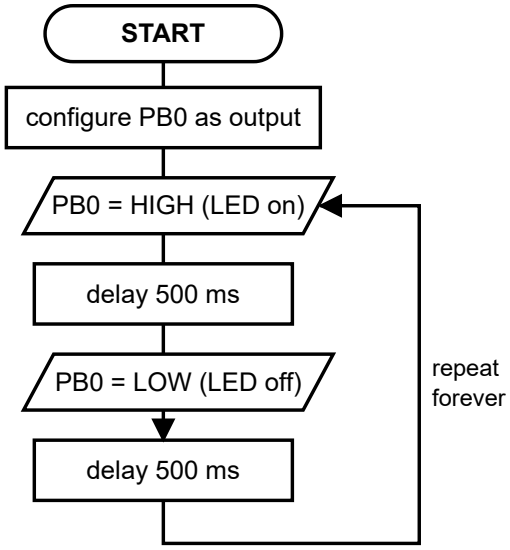
TEST bit 2 (is the pin high?): `if (PIND & (1 << 2))` — AND keeps only that bit

PIND (pin 2 is HIGH)	1	0	1	0	0	**1**	0	0	
mask `(1<<2)` AND	0	0	0	0	0	**1**	0	0	
result	0	0	0	0	0	**1**	0	0	non-zero = TRUE = pin is HIGH. If PIND bit 2 were 0 the result would be 0 = FALSE (pin LOW).

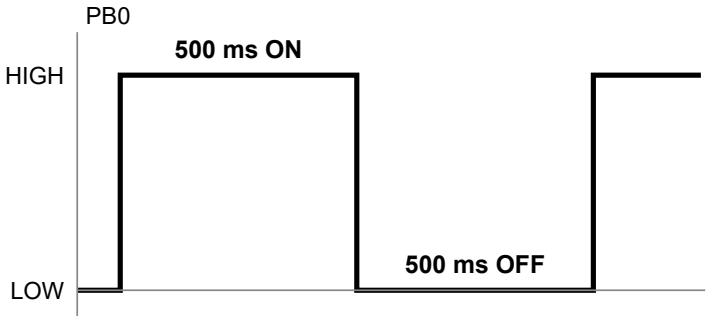
6. System 1: `blinkerAsm` — blink an LED every ½ second (assembly)

An LED on **PB0** (Uno pin 8) through a 220 Ω resistor to GND. The program: configure PB0 as output, then forever: LED on, wait 500 ms, LED off, wait 500 ms.

Algorithm (flowchart)



What the LED does over time



One full blink = 1 second (two delays).
The LED is lit during the HIGH parts.

Figure 5. The firmware is an endless loop: no operating system, so the program must never end.

Why an endless loop? There is no operating system to take control when `main` finishes, so the CPU would run off into empty memory. The loop keeps it busy forever.

The assembly program, line by line

Instruction	What it does
`SBI  DDRB, 0`	**S**et **B**it in **I**/O register: bit 0 of DDRB becomes 1, so **PB0 is an output**. Only bit 0 changes.
`START: SBI PORTB, 0`	`START:` is a label (a name for this address). Bit 0 of PORTB = 1, so PB0 goes **HIGH, LED on**.
`CALL DELAY`	Jump to the `DELAY` subroutine. The return address is saved on the stack so the CPU can come back.
`CBI  PORTB, 0`	**C**lear **B**it in **I**/O register: bit 0 of PORTB = 0, so PB0 goes **LOW, LED off**.
`CALL DELAY`	Wait again with the LED off.
`JMP  START`	Jump back to `START`: the endless loop.

`DELAY: LDI R16, 41`
`LOOP1: LDI R17, 255`
`LOOP2: LDI R18, 255`
	Loa**d** **I**mmediate: put a number into a register. Three counters (R16, R17, R18) for three nested loops. Registers hold 0 to 255 (8-bit).

`LOOP3: DEC R18` `BRNE LOOP3`	`DEC` subtracts 1. `BRNE` = Branch if Not Equal (to zero): jump to LOOP3 while the last result is not 0. This wastes time on purpose.
`DEC R17` `BRNE LOOP2`	After R18 reaches 0, reduce R17 and repeat the inner loop until R17 reaches 0.
`DEC R16` `BRNE LOOP1`	Same for R16, the outermost loop.
`RET`	**Ret**urn from the subroutine to the instruction after `CALL`.

Instruction cycles you need for the delay maths

Instruction	LDI	DEC	BRNE	NOP
Clock cycles	1	1	**2** if it jumps, **1** if it falls through	1 (does nothing)

7. How the delay is calculated (this is exam material)

STEP 1: HOW MANY CLOCK CYCLES EQUAL 0.5 S?

Clock = 16 MHz, so one cycle takes **t = 1 / 16,000,000 = 62.5 ns**.
Cycles needed: **n = 0.5 s / 62.5 ns = 8,000,000 cycles.** So we make the CPU do pointless work that takes 8 million cycles.

LOOP1 (R16 = 41 times)

LOOP2 (R17 = 255 times)

LOOP3 (R18 = 255 times)

```
LOOP3: DEC R18 1 cycle
BRNE LOOP3 2 cycles when jumping, 1 when done
```

n3 = (1 + 2) x 254 + (1 + 1) x 1 = 764 cycles
(254 times it jumps back, the 255th time it falls through)

n2 = (1 LDI + n3 + 1 DEC + 2 branch) x 254 + (1 + n3 + 1 + 1) x 1 = 195,839 cycles

n1 = (1 + n2 + 1 + 2) x 40 + (1 + n2 + 1 + 1) x 1 = 8,029,562 cycles

Figure 6. Three loops inside each other. Each outer loop runs the inner one many times.

THE PATTERN FOR ONE LOOP THAT RUNS N TIMES

cycles = (body + 2) × (N − 1) + (body + 1) × 1
The body includes the `DEC` and, for an outer loop, the `LDI` and everything inside. The branch costs 2 each time it jumps back (N − 1 times) and 1 the last time.

STEP 2 AND 3: FINISH THE CALCULATION

Total cycles = 1 (the first `LDI R16`) + n1 = 1 + 8,029,562 = **8,029,563 cycles**
Delay d = t × n = 62.5 ns × 8,029,563 = 0.50185 s **≈ 0.5 s**

WHY 41 FOR R16?

Cycles for one run of LOOP2 = 195,839. We need 8,000,000 / 195,839 ≈ 40.85, which we round **up** to **41**. If you want it exact, add `NOP` instructions (1 cycle each) to pad the missing cycles.
Why 255? A register is 8-bit (0 to 255). Counting down from 255 gives the longest loop one register can make.
Exercise 1 (slide 36) change: loading R16 with 16 instead of 41 gives about 196 ms ≈ the 200 ms blinker.

Worked exam example (Exercise 2, question 3)

```
DELAY: LDI R16, 0x69      ; 0x69 = 105
LOOP1: LDI R17, 0xFD      ; 0xFD = 253
LOOP2: DEC R17
       BRNE LOOP2
       DEC R16
       NOP
       BRNE LOOP1
       RET
```

Step	Working
Inner loop (LOOP2), 253 times	n2 = (1 + 2) × 252 + (1 + 1) × 1 = 756 + 2 = **758 cycles**
One pass of LOOP1 (jumping back)	LDI 1 + n2 758 + DEC 1 + NOP 1 + BRNE 2 = **763**
The last pass (falls through)	1 + 758 + 1 + 1 + 1 = **762**
R16 = 105 passes	763 × 104 + 762 = 79,352 + 762 = 80,114
Add the first LDI R16	80,114 + 1 = **80,115 cycles**
Convert to time	80,115 × 62.5 ns = 5.007 ms → **≈ 5 ms** (RET and CALL add a few cycles more, negligible)

8. System 2: `blinkerC` — the same blinker in C (200 ms)

Same circuit and PB0. C is easier to write and the compiler turns it into the same kind of machine code.

```
#include <avr/io.h>                 // names like DDRB, PORTB
#define F_CPU 16000000UL             // CPU speed, needed by delay.h
#include <util/delay.h>              // gives _delay_ms()

int main(void)
{
    DDRB |= (1 << PB0);              // make PB0 an output
    while (1)                        // endless loop
    {
        PORTB |= (1 << PB0);         // LED on
        _delay_ms(200);              // wait 200 ms
        PORTB &= ~(1 << PB0);        // LED off
        _delay_ms(200);              // wait 200 ms
    }
    return 0;                        // never reached
}
```

Line	Meaning
`#include <avr/io.h>`	Defines the register names (`DDRB`, `PORTB`, `PINB`, `PB0`…). Without it the compiler does not know them.
`#define F_CPU 16000000UL`	Tells `delay.h` the clock speed (16 MHz) so `_delay_ms` can count the right number of cycles. Must be defined **before** including `delay.h`. **UL** = unsigned long: 16,000,000 is too big for a 16-bit `int` (max 65,535) so you need the suffix.
`int main(void)`	The program starts here (the equivalent of `START:`).
`DDRB |= (1 << PB0)`	Set bit 0 of DDRB: PB0 becomes an output. `PB0` is just the number 0.
`while (1)`	1 is always true, so it repeats forever.
`PORTB |= (1 << PB0)`	PB0 HIGH, LED on.
`PORTB &= ~(1 << PB0)`	PB0 LOW, LED off.
`_delay_ms(200)`	Busy-waits 200 ms (the compiler produces a delay loop, like our assembly).

Slide variants: the first version writes whole values (`DDRB = 0b00000001; PORTB = 0b00000001; PORTB = 0b00000000;`), which overwrite all 8 bits. Fine for one pin; wrong as soon as other pins in the port are in use. The third version uses `|=` and `&=` with `(1<<n)` as above.

9. System 3: `blinker` in the Arduino language

```
void setup()                        // runs once at start
{
  pinMode(13, OUTPUT);              // pin 13 (PB5) is an output
}
void loop()                         // runs again and again
{
  digitalWrite(13, HIGH);           // LED on
  delay(200);
  digitalWrite(13, LOW);            // LED off
  delay(200);
}
```

`setup()` and `loop()` replace `main()` and `while(1)`. Arduino hides the `main` function and calls `loop()` forever for you. Pin **13** is PB5 (the on-board LED), pin 8 is PB0.

Assembly vs C vs Arduino

	Assembly	C	Arduino
Speed / size	Highly efficient	Efficient	Inefficient (bigger, slower)
Ease	Difficult	Easier, libraries	Much easier, libraries
Portable	No (one chip family)	Yes	No (uses the Arduino core)
Risk	Easy to get wrong	Data-type misuse	Hides what the hardware does

WHY IS ARDUINO INEFFICIENT IF IT ALSO USES GCC AND AVRDUDE?

Same compiler, but more code gets linked in. Every Arduino program also contains the Arduino core: `init()` (sets up timers for `millis()`, the ADC, etc.), the generic `pinMode()` and `digitalWrite()` functions that look up which port and bit a pin number means **every time** they run, and checks whether the pin has PWM that must be turned off. Writing `PORTB |= (1<<0)` is a single instruction chosen at compile time.

Evidence from the exercises: chase 5 LEDs: **920 bytes** (Arduino) vs **200 bytes** (C), a 78% saving. Knight Rider: **986** vs **260** bytes. Exercise 2: the C program **142 bytes** vs the Arduino one **743 bytes** for the same job (toggle pin 8).

10. Reading an input: the pushbutton

A micro pushbutton is **normally open**: it connects its two sides only while pressed. We connect it between the pin and **GND**, and switch on the pin's **internal pull-up** (DDR bit = 0, PORT bit = 1). Now the pin always has a defined voltage.

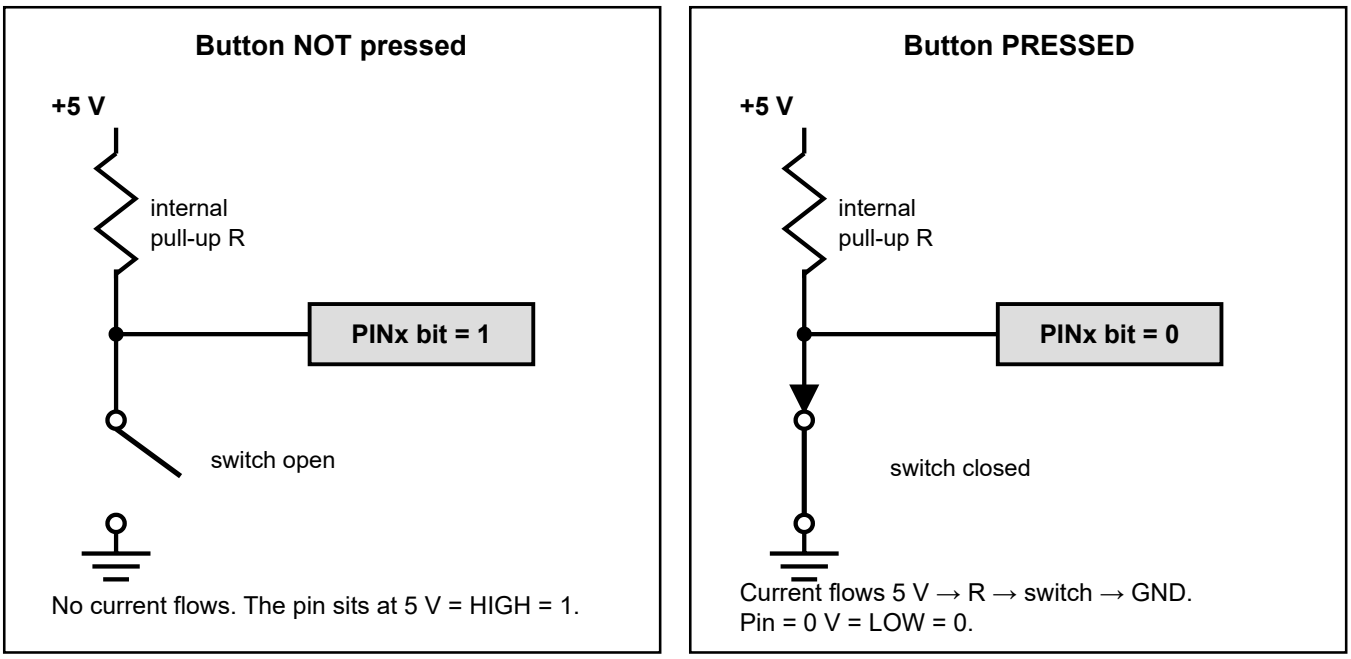


Figure 7. Released = 1. Pressed = 0. The button pulls the pin down to GND against the weak pull-up.

RULE TO REMEMBER

With a pull-up, **pressed = 0 (LOW)**. Released = 1 (HIGH). It feels backwards, but it is the standard way.
To read: `PINx` only. To enable the pull-up: `DDRx` bit = 0 **and** `PORTx` bit = 1. The same `PORTx` register that drives outputs, enables the pull-up on inputs.

11. System 4: `dualRateBlinker`

Task: LED on PB5 toggles every **500 ms** normally, but every **200 ms** while the button on PB0 (Uno pin 8) is pressed. LED on PB5 (pin 13), button from PB0 to GND.

```
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>

int main(void)
{
    DDRB |=  (1 << DDB5);          // PB5 = output (LED)
    DDRB &= ~(1 << DDB0);          // PB0 = input  (button)
    PORTB |= (1 << PORTB0);        // pull-up on PB0 (DDR=0, PORT=1)
    while (1)
    {
        PORTB ^= (1 << PORTB5);    // toggle the LED
        if (PINB & (1 << PINB0))   // PB0 is high = NOT pressed
            _delay_ms(500);          //   slow
        else                         // PB0 is low = pressed
            _delay_ms(200);          //   fast
    }
    return 0;
}
```

Line	Meaning
`DDRB &= ~(1<<DDB0)`	Clear bit 0 of DDRB, input. `~` inverts the mask first.
`PORTB |= (1<<PORTB0)`	On an input, this turns on the pull-up.
`PORTB ^= (1<<PORTB5)`	XOR flips only bit 5: LED on becomes off, off becomes on.

`if (PINB & (1<<PINB0))`	Non-zero (true) when the pin is high = not pressed. So not pressed → 500 ms, pressed → 200 ms.

The Arduino version of the same program

```
const int switchPin = 8;           // PB0
const int ledPin    = 13;          // PB5
boolean state = false;

void setup()
{
  pinMode(switchPin, INPUT_PULLUP);  // input + pull-up
  pinMode(ledPin, OUTPUT);
}
void loop()
{
  state = !state;                  // flip the stored state
  digitalWrite(ledPin, state);     // show it on the LED
  if (digitalRead(switchPin))      // high = not pressed
    delay(500);
  else
    delay(200);
}
```

P. Building circuits by hand (for the practical exam)

In the practical you must build the circuit yourself from a description, with no help. These pages cover the four skills you need: using a breadboard, drawing the schematic, wiring it, and finding faults.

P1. How a breadboard is connected inside

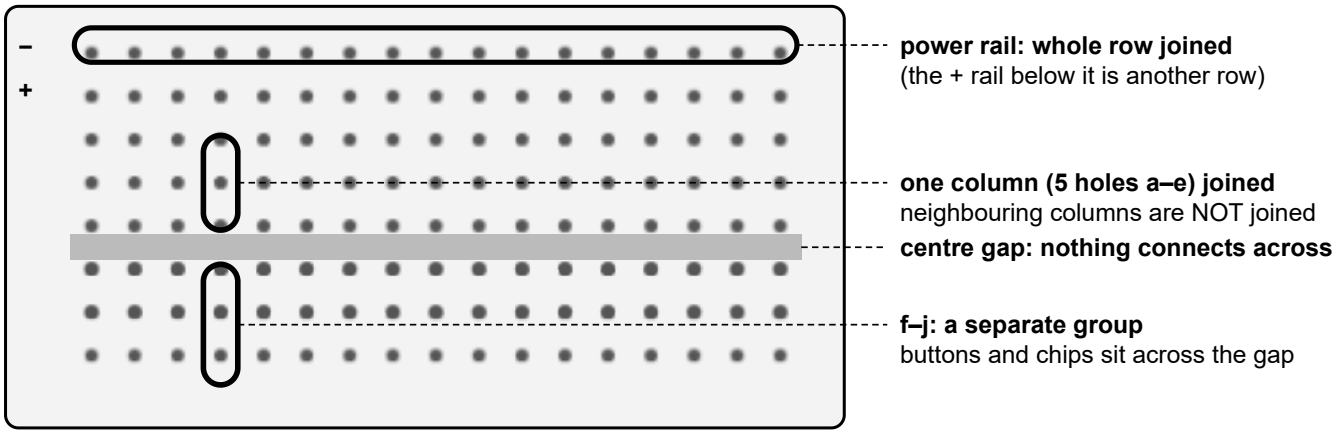


Figure P1. Columns of 5 holes are joined. The long rails are joined along their length (on some long boards the rail is split in the middle; check with a jumper). Chips and buttons sit across the centre gap.

- Two legs in the **same column** are connected. To put two parts in **series** (one after the other), they must share one column and no more.
- Never put both legs of a resistor or LED in the same column: that shorts it out and it does nothing.
- Use the top rail for GND (−) and the next for +5 V (+). Join the Uno's **GND** and **5V** to them first.

P2. Drawing the schematic (exam questions ask for it)

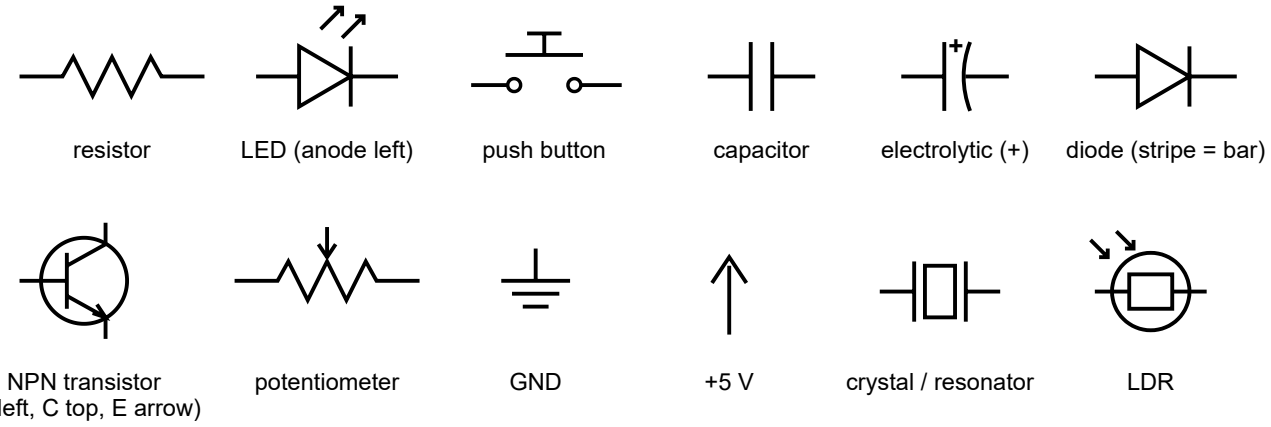


Figure P2. Standard symbols. Draw the chip as a box with the pin names you use (e.g. PB0, PD2), then connect parts to it. Always label values (220 Ω, 1 kΩ, 100 nF) and show +5 V and GND.

EXAM METHOD FOR A SCHEMATIC

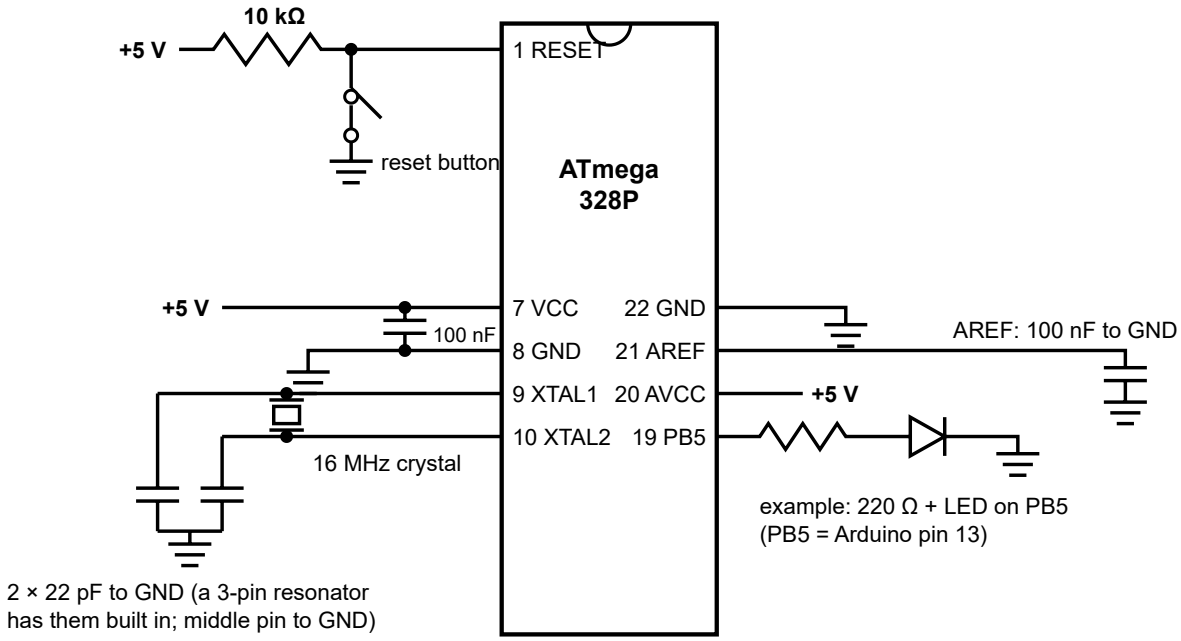
1. Read the spec and list inputs (buttons, sensors) and outputs (LEDs, motor). 2. Pick pins: inputs needing interrupts on PD2/PD3, PWM outputs on 3/5/6/9/10/11, analog on A0–A5, never PD0/PD1. 3. Draw the chip box with only those pins labelled. 4. Each LED: pin → 220 Ω → LED → GND. Each button: pin → button → GND (internal pull-up), plus 100 nF across it if it uses an interrupt. 5. Write the values, then a BOM (bill of materials: list of parts with quantities).

P3. Wiring System 4 (dual-rate blinker) on the Uno, step by step

#	Connection	Check
1	Uno **GND** → breadboard − rail	Every GND later goes to this rail.
2	Uno **pin 13** → a column; 220 Ω from that column to a new column	Resistor legs in two different columns.
3	LED **long leg** in the resistor's second column, short leg in a third column	Long leg = anode, towards the pin.
4	Jumper from the LED's short-leg column → − rail	Closes the loop to GND.
5	Button across the centre gap. Uno **pin 8** → one side; other side → − rail	Use legs on opposite sides of the switch (test 02).
6	USB in, upload, watch	Blinks slowly (500 ms); fast (200 ms) while the button is held.

P4. If the exam says "using an ATmega328P on a breadboard"

Normally you use the chip **on the Uno**, wiring the parts on the breadboard to the Uno's pins. If you must use a **bare chip** on the breadboard, it needs this minimum circuit to run (it is exactly what the Uno board has around the chip, and it is the circuit drawn in every slide schematic):



Both GND pins (8, 22) and both supply pins (7, 20) must be connected.

Figure P3. Minimum circuit for a bare ATmega328P: power on pins 7, 20 (and AREF decoupled), GND on 8 and 22, a 16 MHz crystal on 9 and 10, and a 10 kΩ pull-up on RESET (pin 1) with an optional reset button to GND.

Chip pin ≠ Arduino pin: chip pin 19 is PB5, which is Arduino pin 13. The pin map in section 2 converts between them. A bare chip still needs a way to receive the program: either it already has the Arduino bootloader and you swap it into an Uno to upload, or you use a programmer. Power it from the Uno's 5V and GND pins.

P5. When it does not work: check in this order

1. **Power and ground:** Uno GND joined to the − rail? Every part's GND reaches that rail? (Most faults are a missing GND.)
2. **Columns:** each pair of parts that should connect share a column; no part has both legs in one column; nothing is accidentally across the centre gap.
3. **Direction:** LED long leg towards the pin; diode stripe the right way; transistor C/B/E order.
4. **Pin numbers:** code uses the same pin you wired (pin 13 = PB5, pin 8 = PB0, A0 = PC0).
5. **Code:** DDR set for every output; pull-up for every button; `F_CPU` before `delay.h`; `sei()` if you use interrupts.
6. **Upload:** right board (Arduino Uno) and port; nothing plugged into pins 0 and 1 while uploading.

7. **Divide and test:** upload the plain Blink sketch to prove the LED wiring, or print values with `Serial.println` to see what the code thinks.

> **WRITING PLAIN C IN THE ARDUINO IDE**
> You can write `int main(void)` with registers directly in a `.ino` file instead of `setup()/loop()`. The IDE compiles it as normal C (Exercise 6 asks for this). Do not use `Serial`, `millis()` or `delay()` in that case, because the Arduino `init()` no longer runs; use `_delay_ms()` instead.

12. Getting your program onto the chip



The CPU then reads the instructions from flash one by one. Programming is done once; the firmware stays when the power

Figure 8. From text to running chip.

Option	Notes
Programmer (hardware)	A device that writes the HEX file into the chip. Either a commercial one from the manufacturer or a cheaper home-made or aftermarket one, connected by USB or serial. Used on bare chips.
Self-programming with a bootloader	A small program already stored in the chip's flash (this is what the Uno has). It receives your HEX file over the serial link and writes it into flash itself, so no programmer is needed.
AVRDUDE (AVR Downloader UploaDEr)	The PC program that talks to the bootloader or programmer. Arduino IDE runs it automatically. In Microchip Studio you add it as an external tool. Key options: `-p atmega328p` (chip), `-c arduino` (protocol), `-P COMx` (port), `-b 115200` (baud), `-U flash:w:file.hex:i` (write the hex into flash).

Hardwired alternatives (slide 35)

A blinking LED can also be made with **transistors** or a **555 timer IC** and a few resistors and a capacitor, with no program at all. **Problem:** the behaviour is fixed by the parts. To change the blink rate you must change a resistor or capacitor. With a microcontroller you change a number in the software. That flexibility is why embedded systems moved to software.

13. Exercise 1 solved: chase five LEDs and the Knight Rider

Hardware: five LEDs on **PB0 – PB4** (Uno pins 8 – 12), each through its own 220 Ω resistor to GND. **BOM:** 1 × Arduino Uno, 1 × breadboard, 5 × LED, 5 × 220 Ω resistor, jumper wires.

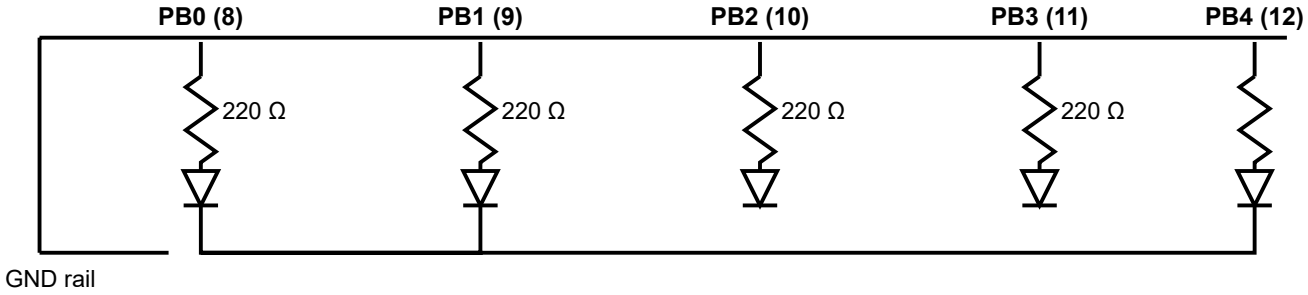


Figure 9. Five identical branches: pin → 220 Ω → LED → GND.

Chase, C (200 bytes)

```
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>
int main(void)
{
  DDRB = 0b00011111;       // PB0-PB4 out
  while (1)
  {
    for (unsigned char i = 0; i <= 4; i++)
    {
      PORTB |= (1 << i);   // LED i on
      _delay_ms(200);
      PORTB &= ~(1 << i);  // LED i off
    }
  }
}
```

Chase, Arduino (920 bytes)

```
void setup()
{
  for (unsigned char i = 8; i <= 12; i++)
    pinMode(i, OUTPUT);
}
void loop()
{
  for (unsigned char i = 8; i <= 12; i++)
  {
    digitalWrite(i, HIGH);
    delay(200);
    digitalWrite(i, LOW);
  }
}
```

Knight Rider, C (260 bytes)

```
  DDRB = 0b00011111;
  while (1)
  {
    for (unsigned char i = 0; i <= 4; i++)
    {                          // left to right
      PORTB |= (1 << i);
      _delay_ms(200);
      PORTB &= ~(1 << i);
    }
    for (unsigned char i = 3; i >= 1; i--)
    {                          // right to left
      PORTB |= (1 << i);
      _delay_ms(200);
      PORTB &= ~(1 << i);
    }
  }
```

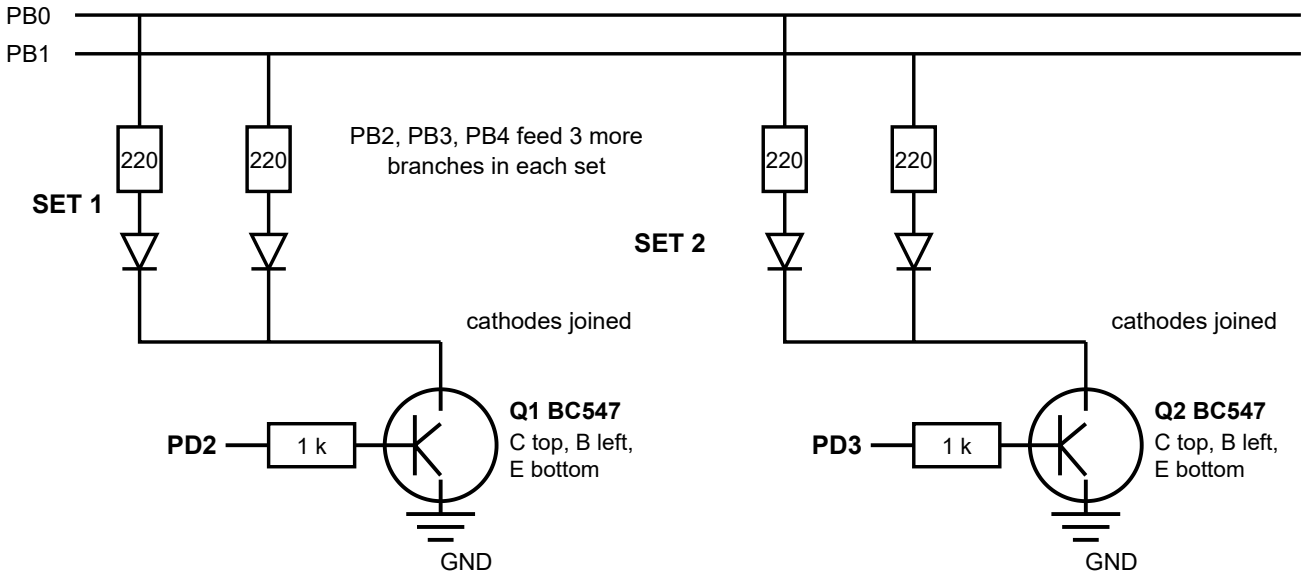
The return trip is 3, 2, 1 so the end LEDs (0 and 4) are not lit twice in a row. The loop `i >= 1` stops safely.

step	LED 0 1 2 3 4
1	● ○ ○ ○ ○
2	○ ● ○ ○ ○
3	○ ○ ● ○ ○
4	○ ○ ○ ● ○
5	○ ○ ○ ○ ●
6, 7, 8	back through 3, 2, 1, then repeat

14. Exercise 1, question 3: ten LEDs on seven pins

Problem: 10 LEDs but a limit of 7 pins. **Idea (multiplexing):** put the LEDs in two sets of five. The **same 5 pins** (PB0 – PB4) feed the anode resistors of **both** sets. Each set's cathodes go to ground through its own BC547 transistor, which acts as a switch. Turn **one transistor on** and only that set can light. 5 + 2 = **7 pins**. The model answer uses **PD2** and **PD3** for the two transistor bases.

PB0-PB4 (5 pins) feed both sets; PD2 and PD3 pick the set



Base HIGH: current flows through the 1 kΩ into the base, the transistor turns on, and that set's LEDs get a path to GND.

Figure 10. Two BC547 transistors switch the ground of each set of five LEDs.

HOW IT WORKS, IN ORDER

1. PD2 HIGH makes a small current flow through the 1 kΩ into the base of Q1 (about (5 − 0.7) / 1000 = 4.3 mA). The transistor turns **on** and connects Set 1's cathodes to GND.
2. PD3 stays LOW, so Q2 is off: Set 2 has no path to GND and stays dark, even though it shares the same anode pins.
3. Raising one of PB0–PB4 now lights the matching LED in **Set 1 only**.
4. To use Set 2, swap: PD2 LOW, PD3 HIGH.

Why the 1 kΩ base resistor? It limits the base current, protecting both the pin and the transistor.
Why PD2 and PD3 and not PB6 / PB7? PB6 and PB7 drive the crystal on the Uno, so they are not available. PB0 – PB4 are already used by the LEDs.

Firmware (C): the model answer, explained

```
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>

void chaseRight(void)
{
    while (!(PORTB & 0b00010000))      // repeat until bit 4 is set
    {
        _delay_ms(200);
        PORTB <<= 1;                   // shift the lit bit 1 place left
    }
}
void chaseLeft(void)
{
    while (!(PORTB & 0b00000001))      // repeat until bit 0 is set
    {
        _delay_ms(200);
        PORTB >>= 1;                   // shift the lit bit 1 place right
    }
}
int main(void)
{
    DDRB = 0b00011111;                   // PB0-PB4 outputs (LED pins)
    DDRD = 0b00001100;                   // PD2, PD3 outputs (transistor bases)
    PORTD = 0b00000100;                  // Q1 on: first set enabled
    PORTB = 0b00000001;                  // LED 0 on
    while (1)
    {
        chaseRight();                    // LEDs 0 to 4 (set 1)
        _delay_ms(200);                  // keep the last LED lit
        PORTD = 0b00001000;              // Q2 on, Q1 off: second set
        PORTB = 0b00000001;              // LED 5 on
        chaseRight();                    // LEDs 5 to 9 (set 2)
        chaseLeft();                     // LEDs 9 back to 5
        _delay_ms(200);
        PORTD = 0b00000100;              // back to the first set
        PORTB = 0b00010000;              // LED 4 on
        chaseLeft();                     // LEDs 4 back to 0
    }
}
```

The shift trick: only one bit of PORTB is 1 at a time. `PORTB <<= 1` moves that bit one place left (LED 0 → LED 1 → …), `>>= 1` moves it right. The `while` loops stop when the lit bit reaches the end (bit 4 or bit 0). The result is 200 ms per LED across all ten, then back.

15. Exercise 2 solved

Question 1: assembly dual-rate blinker (200 ms pressed, 400 ms not pressed)

Assumed wiring: LED on **PB0**, pushbutton on **PD2** to GND (change the names if your paper's figure shows other pins). Two new instructions: `SBIC` = Skip the next instruction if the Bit in the I/O register is Clear. `SBIS` = skip if Set.

```
        SBI   DDRB, 0        ; PB0 output (LED)
        CBI   DDRD, 2        ; PD2 input (button)
        SBI   PORTD, 2       ; pull-up on PD2
START:  SBI   PORTB, 0       ; LED on
        CALL  WAIT
        CBI   PORTB, 0       ; LED off
        CALL  WAIT
        JMP   START

WAIT:   CALL  DELAY200       ; always wait 200 ms
        SBIC  PIND, 2        ; skip next line if PD2 is LOW (pressed)
        CALL  DELAY200       ; not pressed: wait a second 200 ms (= 400 ms)
        RET

DELAY200:                    ; about 200 ms (slide 36 version, R16 = 16)
        LDI   R16, 16
LOOP1:  LDI   R17, 255
LOOP2:  LDI   R18, 255
LOOP3:  DEC   R18
        BRNE  LOOP3
        DEC   R17
        BRNE  LOOP2
        DEC   R16
        BRNE  LOOP1
        RET
```

Logic: pressed (PD2 = 0) → `SBIC` skips the second `CALL DELAY200` → 200 ms. Not pressed (PD2 = 1) → `SBIC` does not skip → 200 + 200 = 400 ms.

Question 2: two programs that look different

```
// Program 1  (C)
#include <avr/io.h>
int main(void)
{
  DDRB = 0x01;
  while (1)
  {
    PORTB = 0x01;
    PORTB = 0x00;
  }
  return 0;
}
```

```
// Program 2  (Arduino)
void setup()
{
  pinMode(8, OUTPUT);
}
void loop()
{
  digitalWrite(8, HIGH);
  digitalWrite(8, LOW);
}
```

Part	Answer
(a)	Both make **PB0 = Arduino pin 8** an output and switch it HIGH, LOW, HIGH, LOW with **no delay at all**. The pin toggles as fast as the CPU can, far too fast for the eye. An LED on pin 8 looks **constantly dim** (about half brightness). On an oscilloscope you would see a very fast square wave (faster for Program 1, slower for Program 2 because `digitalWrite` takes longer).
(b)	The two programs do the same job but the hex files are 142 bytes (Program 1) and 743 bytes (Program 2). Program 2 contains the **Arduino core**: `init()` to set up the timers, the generic `pinMode` / `digitalWrite` code that looks up the port and bit for pin 8 each time (and checks whether the pin has PWM), the hidden `main()`, etc. Program 1 compiles to a handful of direct register instructions. Also Program 2 toggles more slowly because each `digitalWrite` is a function call with those lookups.
(3)	Delay subroutine: **≈ 5 ms**, see the worked example in section 7 (80,115 cycles at 62.5 ns).

L. What the lecturer explained in the lectures (2020 recordings)

These points come from the recorded lectures for this topic (folders 03–08 in `Embedded Systems Lectures`). They fill in what the slides only name. Things he said were exam material are marked **[EXAM]**.

L1. HOW THE PRACTICAL IS MARKED [EXAM]

He said plainly that you **lose marks even if the system works perfectly** when you do not follow standards:

- **Wire colours: red** for +5 V, **black** for GND. Any other colour for power loses marks. (Reason: in big circuits you can only debug fast if you can trust the colours.)
- **Code:** proper variable names (a naming convention, no "garbage" names), **proper indentation**, and **comments**. Missing any of these loses marks "no matter how efficient your algorithm is".

L2. Arduino Uno board tour

Part	What he said
Power jack	Accepts 7–20 V; recommended **9–12 V**. The on-board regulator brings it down to the 5 V the chip needs.
Three ways to power	USB (exactly 5 V), the power jack, or the **Vin** pin (e.g. a 9 V battery to Vin and GND). If several are connected at once nothing breaks: a selector circuit picks one automatically.
Power pins	**5V** and **3.3V** pins to power other parts; two **GND** pins.
Power LED	**The first thing to check when a system does not work.** If it is off: battery, cable, or the power circuit.
USB-to-serial chip	The ATmega328P only speaks **serial**; the PC speaks **USB**. A second small microcontroller (ATmega16U2 on the Uno R3) translates between them. It has its own 16 MHz crystal; the main chip uses a small 16 MHz **ceramic resonator**.
TX / RX pins and LEDs	TX = transmit, RX = receive. To link two boards: TX of one to RX of the other, crossed. The TX/RX LEDs flash when data moves, to confirm communication.
Pin 13 LED	A built-in diagnostic: if a system fails, upload a 1-second blink for pin 13. If it blinks, the board and upload work, so the fault is in your external circuit.
Reset button	Stops the program immediately, clears registers and RAM, and restarts from the first instruction. Use it when the system gets stuck.
Analog pins A0–A5	Analog inputs by default, but can be configured as extra digital I/O.
ICSP header (6 pins)	In-Circuit Serial Programming: for connecting a hardware programmer (L8).

L3. The AVR family, clock and clock sources

- **ATtiny** = small, very low power, few pins, slower. **ATmega** = mid-range (ours). **ATxmega** = high-end, hundreds of I/O pins, more power, expensive. Atmel was later bought by **Microchip**.
- The Uno clocks the chip at **16 MHz**; the manufacturer allows up to 20 MHz. Overclocking beyond that is possible but the chip heats up and needs cooling.
- **Accuracy ranking of clock sources: crystal** (most accurate) > **ceramic resonator** (cheaper, smaller; the Uno's main clock) > **internal RC oscillator** (least accurate, but needs no parts). Without an external clock the chip can run on its internal RC oscillator, and PB6/PB7 become free I/O pins.

L4. Pin multiplexing (one pin, several jobs)

- Port C has only **7** pins (PC0–PC6), not 8.
- **PC6 = RESET**. The bar over RESET means **active low**: the chip resets while the pin is LOW. If you configure the chip not to use reset, PC6 becomes an I/O pin, but then you can only reset by unplugging power.
- **PC0–PC5** = analog inputs or digital I/O. **PD0/PD1** = RX/TX. **PB6/PB7** = external clock, or I/O if the internal clock is used.
- **AVCC** (pin 20) and its GND (pin 22) are a **separate supply for the ADC**; **AREF** (pin 21) supplies the ADC's reference voltage (Lesson 5).
- After reset **DDRx and PORTx are 0x00**: every pin starts as an **input** and every output bit as LOW.

- The `~` (tilde) next to an Arduino pin number marks a PWM-capable pin (Lesson 8).

L5. Electricity details he added

- **Electrons** actually flow from the negative terminal to the positive; by **convention** we say current flows from + to −.
- Current = how much charge flows. Voltage = the "pressure" difference that makes it flow; **no difference, no flow** (same tank height, same pipe = same pressure; wider pipe = more current).
- Resistor **tolerance band** (the separate 4th band): **gold = ±5%**, **silver = ±10%**. A 100 Ω silver may be 90–110 Ω. Precision resistors (1%, 0.5%) cost more. Example: red-yellow-green = 2, 4, ×100 k = 2.4 MΩ.
- **Standard values:** 200 Ω is not sold; choose between 180 and 220 Ω. **Always pick the higher one (220)**: slightly less current is safe; slightly more is not.
- LED: about 2 V and 2–20 mA. Don't call it an "LED light" or "LED diode": LED already means Light Emitting Diode.
- Diode application he highlighted: **reverse polarity protection** (a diode in series stops current if a battery is connected backwards).

L6. The reset pull-up: why the 10 kΩ [EXAM]

RESET is held HIGH through 10 kΩ to 5 V; the reset button pulls it to GND. **Why not just a wire to 5 V?** Then pressing the button connects 5 V directly to GND: a **short circuit**. With the resistor the current is only I = V/R = 5 / 10,000 = **0.5 mA**; without it the current is huge and damages the switch or the supply. A small resistor (100 Ω) is not enough either. He once gave a **partially completed schematic containing deliberate errors** in the exam and asked students to complete and correct it, and this was the error. **Schematic convention:** a **dot** where wires cross means they are joined; no dot means they just cross.

After programming, a product needs only: the chip, the clock source, the LED + resistor (whatever the task uses), and optionally the 10 kΩ + reset button. The USB chip, regulator and indicator LEDs are only for programming and convenience. You can program the chip on the Uno, then move it to your own board.

L7. Breadboard, and why it's called that

Inside: the top two and bottom two rows are joined **horizontally** (power rails); the middle columns are joined **vertically**, with a break in the middle. "You must keep this pattern in your mind." Early engineers soldered every prototype, which was slow to change, so one used his mother's wooden bread board with nails to connect parts quickly; the name stuck. **Stripboard / dotboard** ("Vero" is a brand) is the next step: parts soldered on a board with copper strips or dots, before a PCB.

L8. Why embedded programs have no end [EXAM concept]

On a PC, when your program ends, control returns to the **operating system**. A microcontroller has **no OS**, and the CPU's fetch–execute cycle can never stop. So if the program reached its last line, execution would "fall out" into empty memory. Therefore: **flowcharts have no END node, and an endless loop is a must** (`while(1)`, `JMP START`, Arduino's `loop()`). In PC programming an endless loop is a bug; here it is required. Like a washing machine that repeats its cycle until switched off.

L9. Delay loops, assembly and the tools

- **Assembly is not a separate language**: it is the processor's own instruction set written as opcode + operands. The instruction set summary is at the end of the datasheet, with the clock cycles of each instruction. Most AVR instructions take 1 cycle (RISC).
- A delay is just a loop that does nothing for a number of cycles. Even `NOP` takes a cycle: the CPU never stops.
- **One 8-bit loop is far too short:** at most 256 iterations × 62.5 ns ≈ 16 µs, so nested loops are needed. Use letters (X, Y, Z) for the loop counts and the formula works for any values.
- **[EXAM]** "For the last three years I gave a delay calculation in the exam." A calculator is allowed.
- Real delay libraries compute the loop counts from the requested time with an algorithm; the nested-loop method shows the principle.
- **Microchip (Atmel) Studio:** New Project → AVR Assembler (or GCC C Executable) → **select the device ATmega328P**. Selecting the device adds `m328pdef.inc`, which maps names like `DDRB` to their **memory addresses**. Build → Build Solution makes the `.hex` in the Output Files folder. Syntax errors are caught; logic errors are not.
- Blinker sizes: **assembly 38 bytes** vs about **936 bytes** in Arduino. An **oscilloscope** (voltage vs time on screen) is used to measure the real timing.

L10. Ways to program the chip (and how a bootloader works)

Option	Details
Manufacturer programmer (COTS)	e.g. Atmel ISP. **ISP = In-System Programming**: the chip is programmed while in the circuit (in the 90s you had to remove it). Connects to the 6-pin **ICSP** header. Reliable, USB, but about $60.
Aftermarket / homemade	Cheap USB programmers (eBay) via ICSP; homemade **serial** programmers are easy to build but PCs no longer have serial ports.
Self-programming (bootloader)	What Arduino uses; an AVR innovation (before AVR, you always needed a hardware programmer).

BOOTLOADER, STEP BY STEP

Flash is split into a **bootloader section** and an **application section**. A **bootloader** is a small program **pre-burned** into the bootloader section (once, with a hardware programmer). A **host application** on the PC (**AVRDUDE** = AVR Downloader UploaDEr) talks to it.

1. On every power-up or reset the bootloader runs first (its first instruction is at address 0). 2. It talks to the host over serial (through the USB-serial chip): "is there a new application?" 3. If yes, it writes the new HEX into the application section and jumps to it. 4. If no, it jumps to the application already there.

Two drawbacks [EXAM]: (1) it **uses flash space** (e.g. a 2 KB bootloader leaves 30 KB of 32 KB, so a 31 KB program no longer fits); (2) a **start-up delay** on every power-up while it checks for new code, which matters for time-critical systems. Advantage: no programmer, one click to upload.

L11. AVRDUDE arguments (to add it as an external tool in Microchip Studio)

AVRDUDE is free, open source and installed with the Arduino IDE (`...\Arduino\hardware\tools\avr\bin\avrdude.exe`). In Microchip Studio: Tools → External Tools (set the profile to **Advanced** if the menu is missing). Type it exactly: **one extra space breaks it**.

Argument	Meaning
`-C "...\etc\avrdude.conf"`	path of AVRDUDE's configuration file
`-v` / `-v -v`	verbose: more `v` = more detail in the output window
`-p atmega328p`	the part (chip)
`-c arduino`	programmer type
`-P COM3`	the board's port: find it in Windows **Device Manager** → **Ports**
`-b 115200`	baud rate the bootloader uses
`-U flash:w:"$(TargetName).hex":i`	memory **flash** : operation **w**rite : file : format **i** = Intel HEX. `$(TargetName)` / `$(TargetDir)` are generic names, so you never type each project's file name.

Success ends with "**avrdude done. Thank you.**" and shows the bytes written. `avrdude -?` lists every option. The Arduino IDE runs the very same tool.

L12. Hardwired vs software-based (exam discussion question)

An LED can blink with **two transistors** (astable multivibrator) or a **555 IC**; the rate is set by R and C. Software is better because it is: **flexible** (change one line, e.g. 41 → 16 for 200 ms); **easy to modify, debug and maintain**; **updatable remotely** (firmware updates, even over the internet, with no product recall); **cheaper in the long run**; and the same generic hardware can become different products. That is why industry moved from hardwired to software.

L13. C details he stressed

- `avr/io.h` is like a **dictionary**: it tells the compiler that `DDRB` is a certain memory address.
- Without `F_CPU` the delay library **assumes 1 MHz**, so on a 16 MHz chip every delay runs about **16 times too short** and the timing is wrong. Define `F_CPU 16000000UL` **before** including `delay.h`. **UL** = unsigned long, because the compiler treats a plain number as `int`, which cannot hold 16 million.
- **A home-made delay loop** `for (long i = 0; i <= 95000; i++);` **does not work**: compiler **optimisation** sees it does nothing and removes it. It works only with optimisation disabled. Optimisation also does things like replacing a never-changed variable with its constant value.

- **Choose the smallest data type:** student marks (0–100) → `unsigned char` (0–255, 8 bits), not `int` (16 bits on AVR, 32 on a PC), `float` (32 bits) or a string. `char` is just an 8-bit number.
- C gives the programmer full control and full responsibility: no array bounds checking, no automatic memory freeing (memory leaks). Powerful, but mistakes are yours.
- C is **portable**: the same source compiles for AVR, PIC or a PC with the right compiler. Assembly and Arduino code are not.

L14. Arduino under the hood, and when to use it

- **Arduino is not a language**: it is a hardware/software platform. The language is C/C++; the compiler is GCC; the uploader is AVRDUDE. It was designed for non-technical users (e.g. fashion-design students, hobbyists), so friendliness was chosen over efficiency.
- **Wrapper functions** ("sugar-coated medicine") hide the registers. `pinMode` calls `digitalPinToBitMask`, `digitalPinToPort`, `portModeRegister`, `portOutputRegister`…; `digitalWrite` also checks and turns off PWM, because the same function must work on Uno, Mega, Due, NodeMCU and more. Source: `...\hardware\arduino\avr\cores\arduino\wiring_digital.c`.
- The hidden `main.cpp` calls `init()`, then `setup()`, then `for(;;) loop();` forever.
- `digitalWrite(8, HIGH);` can be replaced by one register line, `PORTB |= (1<<0);`.
- **Efficiency ranking:** all assembly > all C > a mix of Arduino + C + inline assembly. But first make the **algorithm** efficient: bad assembly is still bad.
- **Practical route (e.g. a hackathon robot):** prototype quickly in Arduino → replace slow wrapper calls with register C → rewrite the most frequently called functions in **inline assembly** (`asm` inside C) → improve the algorithm. Like the memory hierarchy: speed where it matters most, within the time available.
- "Just buy faster hardware" (e.g. Due at 84 MHz instead of Uno at 16 MHz) is the Arduino fans' argument. His counter: efficient firmware lets you use **cheaper** hardware. A good driver doesn't need a Ferrari.
- This course uses the **middle path**: C with direct register access.

L15. Other Arduino problems: licence, debugging, cost

Issue	Detail
LGPL licence (Lesser General Public License)	Free and open source, usable in free or commercial products, with conditions: (1) hardware **derived from an Arduino board design** → you must **release your hardware design**; (2) firmware using the Arduino core/libraries → you may keep the source but must release the **object file** so users can relink it with newer libraries, and object code can be reverse-engineered; (3) if you **modify** the core or a library → you must **release your source**.
No debugger	Debugging means adding `Serial.print` lines, and each one **changes the timing** of a time-critical system.
Board cost and power	The on-board USB-serial chip makes the board more expensive and power-hungry; a mass-produced product does not need it after programming.

L16. Measured: how much slower is Arduino?

Toggling pin 8 in `loop()` / `while(1)`	Frequency	Period
`digitalWrite(8, HIGH); digitalWrite(8, LOW);`	135 kHz	7.38 µs (HIGH 3.5 µs, LOW 3.9 µs)
`PORTB = 0b00000001; PORTB = 0b00000000;`	4 MHz	250 ns

Improvement = (7380 − 250) / 7380 × 100 = **96.6%**. **Why is the LOW part longer?** After the LOW write, the loop also has to jump back and check the loop condition before the next HIGH, and that extra time is added to the LOW part. **Code size, dual-rate blinker:** assembly **48** bytes, C **190**, Arduino **1086**. With single-cycle instructions assembly can toggle at the full 16 MHz; a 2-cycle instruction halves that.

L17. Bitwise points he made

- **Why shifts instead of binary strings?** With 8 bits you can count them; with **32-bit** registers (what industry uses) counting 32 ones and zeros is error-prone. `(1<<n)` avoids it.
- **Shift = fast maths:** left shift by n = multiply by 2^n; right shift by n = divide by 2^n (8 >> 2 = 2). Shifting is much cheaper for hardware than a multiplier. Bits shifted past the end are **lost**: in 8 bits, 16 << 4 = 0 (overflow).
- **Label constants** (`DDB5`, `PORTB0`, `PINB0`) are defined in `iom328p.h`, which `io.h` includes for you. `DDB5` is just 5. They make code readable.

- **[EXAM]** Know all three styles (binary strings, shifts with numbers, shifts with labels) because "in the exam I may ask you to explain a code segment" in any of them.
- Three ways to toggle in Arduino: a `boolean state = !state`; an `int state ^= 1`; or `digitalWrite(13, !digitalRead(13))`.
- In C, an `if` is false only when the expression is **0**; any other value is true. So `if (PINB & (1<<0))` works without `== 1`.

L18. Two small practical tips

- **Micro switch legs:** legs A–C are always joined, and B–D are always joined. Use **diagonal** legs (A and D, or B and C) and you always get the switch action.
- **10 LEDs on 7 pins (assignment hint):** split into two groups of five on the same 5 pins; give each group its GND through its own BC547 (1 kΩ base resistor); turn on one transistor at a time (section 14).

16. Common mistakes and quick checks

Mistake	What goes wrong / the fix
Forgot to set `DDR`	Pin stays an input. The LED never lights or is very dim. Always configure direction first.
`PORTB = ...` instead of `|=` / `&=`	Overwrites all 8 pins, switching other LEDs or pull-ups off. Use set/clear/toggle patterns.
Forgot `~` when clearing	`PORTB &= (1<<0)` clears everything except bit 0. You need `~(1<<0)`.
No pull-up on a button input	The pin floats, reads random values when the button is open. Set `PORTx` bit = 1 on the input.
Thinking pressed = 1	With a pull-up to GND, pressed = 0.
`F_CPU` missing or after `delay.h`	The delay is wrong or the compiler warns. Define it first, with `UL`.
LED without a resistor	Too much current. Use 220 Ω (or compute $(V_s - V_L)/I$).
Mixing Arduino pin numbers and port bits	Pin 13 = PB5. Pin 8 = PB0. A0 = PC0. Keep the map handy.
Using pins 0 and 1 for parts	They are the serial pins (RX, TX). They conflict with uploading and USB.
No endless loop in `main`	The CPU runs off the end of the program. Always `while(1)`.

Cheat sheet: what is NOT on the exam sheets

Operators, delay functions, Arduino functions and register bit names are on the three sheets you get in the exam. This page is only the rest.

Registers (x = B, C, D)	
DDRx	1 = output, 0 = input
PORTx	Output: 1 = HIGH, 0 = LOW Input: 1 = pull-up on
PINx	Read the pin (1 = HIGH)

Bit patterns, bit n	
Output	`DDRx |= (1<<n);`
Input	`DDRx &= ~(1<<n);`
Pull-up	`PORTx |= (1<<n);`
HIGH	`PORTx |= (1<<n);`
LOW	`PORTx &= ~(1<<n);`
Toggle	`PORTx ^= (1<<n);`
Read	`if (PINx & (1<<n))`

Assembly	
`SBI / CBI reg,n`	set / clear bit n of an I/O register
`LDI Rd,k`	load number k (1 cycle)
`DEC Rd`	subtract 1 (1 cycle)
`BRNE label`	jump if last result ≠ 0 (2 / 1 cycles)
`CALL / RET`	subroutine call / return
`JMP label`	jump (endless loop)
`SBIC / SBIS`	skip next if bit clear / set

Formulas	
Resistor	$R = (V_s - V_{LED}) / I$
Cycle time	t = 1 / f = 62.5 ns at 16 MHz
Cycles for delay	n = d / t
Loop (N times)	(body+2)(N−1) + (body+1)
Delay	d = t × n

PIN MAP

PD0–PD7 = pins 0–7 | PB0–PB5 = pins 8–13 (PB5 = on-board LED) | PC0–PC5 = A0–A5 | PWM pins: 3, 5, 6, 9, 10, 11 | Avoid pins 0 and 1.